INTEGRATION, the VLSI journal 97 (2024) 102193

Contents lists available at ScienceDirect

Integration, the VLSI Journal

journal homepage: www.elsevier.com/locate/vlsi

Enhancing cell delay accuracy in post-placed netlists using ensemble tree-based algorithms

Yassine Attaoui [a,*], Mohamed Chentouf [a], Zine El Abidine Alaoui Ismaili [b], Aimad El Mourabit [c]

[a] Siemens EDA/CSD Calypto - Synthesis Solutions, Rabat, Morocco
[b] Information, Communication and Embedded Systems (ICES) Team, Mohammed V University, Rabat, Morocco
[c] System & Data Engineering Team, ENSA of Tangier, Abdelmalek Essaâdi University, Tangier, Morocco

ARTICLE INFO

Keywords:
Delay estimation
Machine learning
Tree-based algorithms
Post-placement delay estimation
Cell delay

ABSTRACT

Nowadays, the ASIC design is increasing in complexity, and PPA targets are pushed to the limit. The lack of physical information at the early design stages hinders precise timing predictions and may lead to design re-spins. In previous work, we successfully improved timing prediction at the post-placement stage using the *Random Forest* model, achieving 91.25% cell delay accuracy. Building upon this, we further investigate the potential of *Ensemble Tree-based* algorithms, specifically focusing on "*Extremely Randomized Trees*" and "*Gradient Boosting*", to close the gap in cell delay accuracy. In this paper, we enrich the training dataset with new 16 nm industrial designs. The results demonstrate a substantial improvement, with an average cell delay accuracy of **92.01%** and **84.26%** on unseen data. The average Root-Mean-Square-Error is significantly reduced from **12.11** to **3.23** and **7.76** on unseen data.

1. Introduction

Recently, there has been a growing focus on achieving high timing accuracy during early design stages. The behavior of post-placement gate-level timing becomes particularly crucial as it directly affects the subsequent place-and-route stages. Timing inaccuracies during the placement stage can have significant implications on static timing analysis (STA), potentially leading to false reports of timing violations and subsequent functional failures. These inaccuracies can result in sub-optimal clock network designs, excessive power consumption and clock skew. As per the routing process, the delay inaccuracy may lead to lengthened or shortened wires, thus impacting design power and causing design re-spin as well as increasing design costs.

Achieving timing closure in the early stages is crucial and various pre-placement timing optimization techniques are employed to enhance timing and alleviate timing critical paths. These techniques encompass logic reconstruction, timing-driven synthesis, technology mapping, gate-sizing, buffering, and clock-gating, among others, all aim to reduce delays on critical paths and ensure compliance with timing constraints [1–3]. Nevertheless, these techniques are effective in improving timing at the current design stage, however, they may not fully considering the impact on subsequent stages and overall design performance.

Additionally, the lack of physical information at early stages can result in over-pessimism which can cause sub-optimal routing and congestion. This, in turn, causes the EDA tools to miscalculate the physically related metrics such as wire length, capacitive loads, slew, and RC parasitics. While the imprecision in cell and interconnect delays diminishes as we progress through the physical design process and the layout information is no longer estimated, enhancing timing predictions at an early stage remains essential.

The primary source of inaccuracy in timing estimations comes mainly from the lack of routing information. In Fig. 1, we showcase an example of post-placed cell delays for a 16 nm commercial design. The blue dots represent *Oasys-RTL* delay estimations difference from the accurate delay obtained from *Aprisa P&R*. We realize clearly the worst estimations deviate significantly exceeding 150 ns. Such misestimations can result in an over-designed layout, leading to unnecessary power and area consumption. As accurate timing information is available till after detailed routing, the estimations made during previous stages affect the design quality and functionality.

Recent studies tend to apply machine learning algorithms on downstream stages by training models using a physically aware netlist and then using the trained model to attenuate the inaccuracy of estimations in upstream design stages. The focus of this paper is to enhance the timing accuracy during the post-placement stage by leveraging machine learning Ensemble Tree-based algorithms. We attempt to improve cell delay accuracy of *Oasys-RTL Synthesis tool (Siemens EDA)* [4] for post-placed gate netlist based on equivalent netlist generated using physical

* Corresponding author.
E-mail address: yassine.attaoui@siemens.com (Y. Attaoui).

https://doi.org/10.1016/j.vlsi.2024.102193
Received 25 August 2023; Received in revised form 16 February 2024; Accepted 27 March 2024
Available online 1 April 2024
0167-9260/© 2024 Elsevier B.V. All rights reserved.

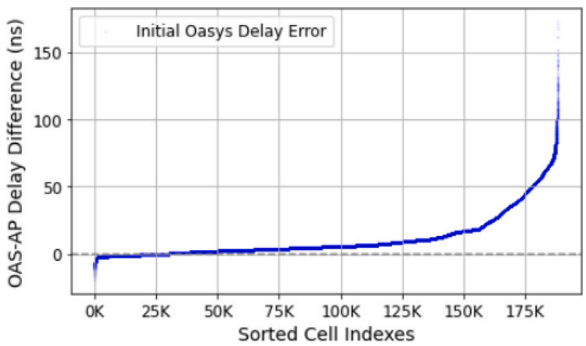

Fig. 1. Analyzing Oasys delay inaccuracy in comparison to Aprisa cell delays.

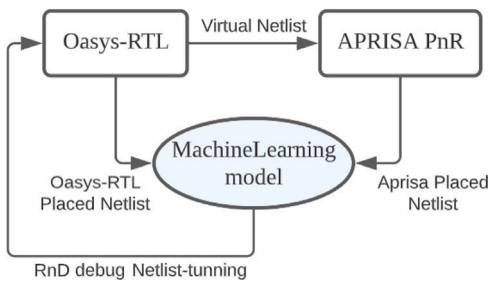

Fig. 2. ML-based Oasys-RTL timing improvement.

aware P&R tool *Aprisa PnR (Siemens EDA)* [5]. Our approach consists of training various *Supervised* regression models on a set of 16 nm industrial designs to close the error gap on cell delays. To ensure fair comparison, we strive to keep the similarly netlists, although every tool employs its own placement and optimization techniques.

To commence, we create the *Oasys Virtual-Netlist*, which is subsequently input to *Aprisa* for additional enhancements. Concurrently, Oasys executes its own optimizations to produce the *Oasys Final Netlist*. Eventually, we extract cell delays from both the final Oasys netlist and the Aprisa post-placed netlist, as depicted in Fig. 2.

The rest of this paper is organized as follows. Section 1 provides an overview of the current state-of-the-art techniques for improving timing accuracy in VLSI design using machine learning, along with a review of related works in this area. In Section 2, we describe the dataset and flow used and give a brief background on ML algorithms employed, and outline the methodology we followed for data processing, model training, testing, and evaluation. In Section 3, we present the results of our experiments. Finally, in Section 4, we draw conclusions based on our findings and discuss potential future research.

2. Related work

Machine learning has increasingly been applied to timing prediction, as evidenced by recent research. This section reviews various learning -based strategies for timing improvement, differentiating our method from existing state-of-the-art approaches.

[6] developed a learning models using ANN, SVR, and Random Forest to rectify timing discrepancies in 28 nm and 45 nm designs, the study aims to reduce timing divergences in flip-flop setup time, cell arc delay, wire delay, stage delay, and path slack at timing endpoints. [7] introduced models employing Lasso, Neural Networks, and Random Forest for pre-routing timing prediction, significantly reducing timing pessimism in 45 nm designs' net delays. The models were validated on 20 circuit implementations, with up to 45K nets. [8] focused on reducing net delay divergence and slew uncertainty using a learning-based model to correlate with signoff STA tool estimations. The model, trained on features like slew transition time, actual/required arrival times, and cell/wire delay, was tested on designs ranging from 30K to 150K instances, demonstrating potential in mitigating misestimations.

In [9], the study explores a predictive timing model, specifically correlating GBA and PBA results. It uses CART to predict PBA arrival times and slacks, utilizing a dataset of 1.7 million samples with 13 features from GBA. The dataset comprises in-house artificial designs and five medium-sized real designs. The research highlighted that dropping one feature led to a 2% drop in accuracy, while dropping two features resulted in a 4% decrease, indicating that incorporating more features could enhance accuracy. Our study aims to find a balance between maximizing the number of features and minimizing runtime, thus achieving a lower runtime with a higher number of features.

[10] introduces a learning-based approach for detecting memory timing failures from the pre-placement stage, utilizing models like Lasso, SVM, Gradient Boosting, ANN, and HSM to predict post-layout slack. The study analyzed 7 small-sized 28 nm netlists, each not exceeding 200K instances, and training models on 20 features derived from netlists, floorplans, and constraints. [11] predicts crosstalk-prone nets, coupling capacitance, crosstalk noise, and incremental delay using algorithms such as LR, ANN, RF, and XGBoost to improve design routability and timing at the routing stage, focusing on 12 benchmark designs at the 7 nm, with instances ranging from 3.5K to 74K. The models, trained on 20 net features including topology, wire delay, slew, and electrical/logical aspects like capacitance and fan metrics, found XGBoost to be the most accurate, achieving around 87% accuracy in incremental delay prediction.

In [12], the author developed a model to improve switching power estimation using back-annotated RC parasitics from SPEF files, aiming for a "Spefless flow" that predicts cell switching power without relying on SPEF files. The model was trained with 80% of data from a single test, which might limit its generalization across different designs. Utilizing Linear Regression, Random Forest, and Decision Tree algorithms, the model, despite limited training data, showed high accuracy and significantly minimized errors in switching power predictions.

In [13], a pre-routing timing prediction employing Graph Neural Networks (GNN) was developed to enhance net delay and pin arrival times without the need for external router and timer engines. The study utilized 21 benchmarks from the 130 nm library, dividing them into 14 for training and 7 for testing, with a total training dataset of 920K samples, achieving an average accuracy of 94%. Similarly, [14] developed an ANN and SVM-based models to predict slew, incremental delay, and path delay related to Signal Integrity (SI), tackling timing discrepancies between SI-mode and non-SI-mode. The models were trained on a dataset of 6 designs, with instance sizes ranging from 6.5K to 350K. [15] emphasizes the impact of timing optimization on the pre-routing timing estimations. The model introduced combines Graph Neural Network (GNN) and Convolutional Neural Network (CNN) to integrate layout and netlist information. The GNN model extracts and aggregates netlist information for each endpoint, whereas the CNN discerns specific layout characteristics for each endpoint. This strategy allows the net-cell delays prediction following post-timing optimization and netlist restructuring, including after modifications and re-labeling. The prediction accuracy is assessed by comparing it the sign-off delays, achieving an average R2 score of 87%. [16] introduced a post-placement net delay predictor that generates a look-ahead RC network. This Random Forest-based model is trained on cell and net-related features to accurately predict parasitic resistance, capacitance, net delays, and critical path slack. The results show a strong correlation with pre-route sign-off STA results. [17] addressed gate delay discrepancies between single-input switching (SIS) and multi-input switching (MIS). Employing Polynomial Regression, SVR, and ANN, the study trained models on gate attributes such as output load and slew. The models were then integrated into the timing library for timing corrections. The ANN model has significantly reduced delay misestimations from 120% in SIS to 3%.

While prior research on early-stage timing improvement predominantly focused on small to medium-sized designs from higher technology nodes (28 nm and 45 nm), exploration of larger designs has been limited. These studies often utilized methods such as Random Forests, Gradient-based models, and Neural Networks, achieving high prediction accuracy but generally depending on a restricted set of features, approximately 20 parameters, which could potentially affects the accuracy. In contrast, our study encompasses 7 designs from the 16 nm technology node, with most designs surpassing 430K instances, demonstrating a significant increase in scale and complexity. Our training set, exceeding 2.3 million samples, marks a considerable extension beyond previous studies, leading to an enhanced cell delay accuracy of around 92%.

Building upon our previous work [18], which primarily utilized the Random Forest model to enhance timing accuracy in both pre-placement and post-placement stages, our current study adopts advanced tree-based algorithms. These include Extremely Randomized Trees (Extra-Trees), Gradient Boosting (GB), Light Gradient Boosting (LightGBM), eXtreme Gradient Boosting (XGBoost), and Adaptive Boosting (AdaBoost). Moving forward, we plan to focus on deep learning neural networks and introducing new design methodologies, which will be elaborated in an upcoming publication. This work contributes several key advancements:

- This work is the first of its kind to predict post-placement cell delays for large 16 nm industrial designs with diverse implementations.
- Utilizing a comprehensive set of 45 cell and net attributes, the models are trained to accurately capture cell behavior and characteristics.
- The predictions are based on timing arcs, cell unateness and arc sense attributes.
- The model parameters are carefully tuned to optimize their performance, leading to the selection of the best hyperparameters.
- The proposed models demonstrate strong generalization capabilities, effectively handling unseen data and maintaining their accuracy.

3. Designs characterization and ML algorithms

Our approach consists of correcting Oasys-RTL cell delay estimation by leveraging multiple Tree-based machine-learning algorithms. Our objective is to compare different Ensemble Tree-based models using a carefully cleaned dataset. Before training and testing the models, we preprocessed the dataset by removing outliers and fine-tuned the hyperparameters for each model to identify the parameters that contribute to the best estimation performance. In this section, we present the workflow we followed, including a detailed description of the dataset composition and the extracted features.

3.1. Methodology

Our methodology consisted of executing two distinct workflows: one employing the *Oasys-RTL Synthesis* tool, and the other utilizing Aprisa P&R. Fig. 3 illustrates the process followed in each of these flows. In the Oasys-RTL flow [4], we started by feeding the design's RTL database, constraints, and predefined floorplan. The tool then performed synthesis, technology mapping, and timing-area optimizations. Afterward, the Synthesis tool generates a virtual unplaced gate netlist. In Aprisa P&R flow [5], we used the previously generated virtual unplaced netlist as input. Aprisa P&R continue with additional placement and attractively optimizing the design until closing timing. To minimize netlist modification, we instructed Aprisa to perform placement with low effort. Finally, we extracted cell and net delays, and various cell and net attributes, from both tool. All the collected features, including cell delays per timing arcs and other metrics, were

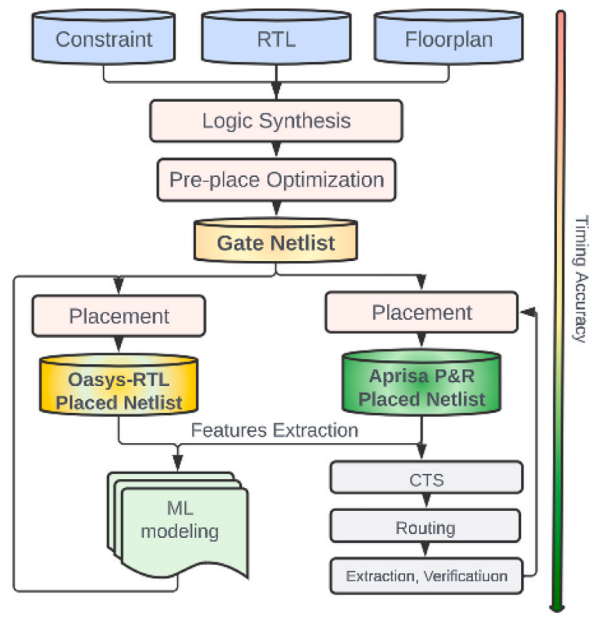

Fig. 3. Oasys Aprisa placement flow.

stored in a CSV file. Furthermore, we have discarded net delays during this study and it will be addressed in a separate paper. For the purposes of our analysis, We directed both tools to execute using the parameters "0-RC" and "0-LOAD" sequentially.This configuration assumes perfect interconnects with no delays attributed to the nets, enabling us to concentrate exclusively on the cell delays and their associated timing arcs.

3.2. Initial timing discrepancies

The presence of physically aware parameters such as wire length, resistance, and capacitance, makes *Aprisa* trustworthy. As we aim to improve the convergence of delays at the post-placement without running the full physical design steps, we minimized the netlists differences. With the help of multiple tree-based algorithms, we conducted the training using a large dataset built from six 16 nm designs of different sizes, as detailed in Table 1. The total number of standard cells in each design reflects its size. Initial delay discrepancies between Oasys and Aprisa are visualized in scattered plots as in Fig. 4. The plots reflect the Oasys-RTL cell rise delay and Aprisa cell rise delay per design, as a matter of space we did not include the fall delays figures. The plotting includes a heatmap which refers to the cell's output capacitance. In the legend, we included the initial Oasys and Aprisa Mean and Standard deviation.

3.3. Data pre-processing

In this paragraph, we describe the data pre-processing steps performed before training models. Our data collection process involved generating a series of CSV files from both tools. The CSV files encompass a wide array of crucial information, including cell and net properties, the cell-net mapping connections, cell library attributes, and various physical metrics like cell input/output transitions, output capacitances, and delays. By employing an inner join on the cell's *full name*, *from pin* and *to pin* columns, we effectively consolidated all the individual CSV files into a unified master CSV. These specific columns were designated as indexes (IDs), serving to uniquely identify each timing arc. As part of the data preparation process, certain columns that exhibited limited informative value for our predictive purposes were excluded from the master CSV.We leveraged the Scikit-learn,

Table 1
Designs characteristics.

Design	Node (nm)	Clocks#	Freq. (Ghz)	Std cells#	Macro#	Area (squm)	FF#	Comb#
TC1	16	2	2	**2274**	0	1100	413	1611
TC2	16	2	1.4	**30 364**	0	19 134	9424	20 496
TC3	16	2	1.6	**38 826**	0	27 297	12 146	24 567
TC4	16	2	0.52	**75 144**	0	44 501	19 388	55 236
TC5	16	2	1	**430 975**	0	212 149	49 824	362 564
TC6	16	18	1.9	**775 295**	0	336 174	97 119	633 362
TC7	16	2	1	**589 447**	0	275 677	86 651	478 745

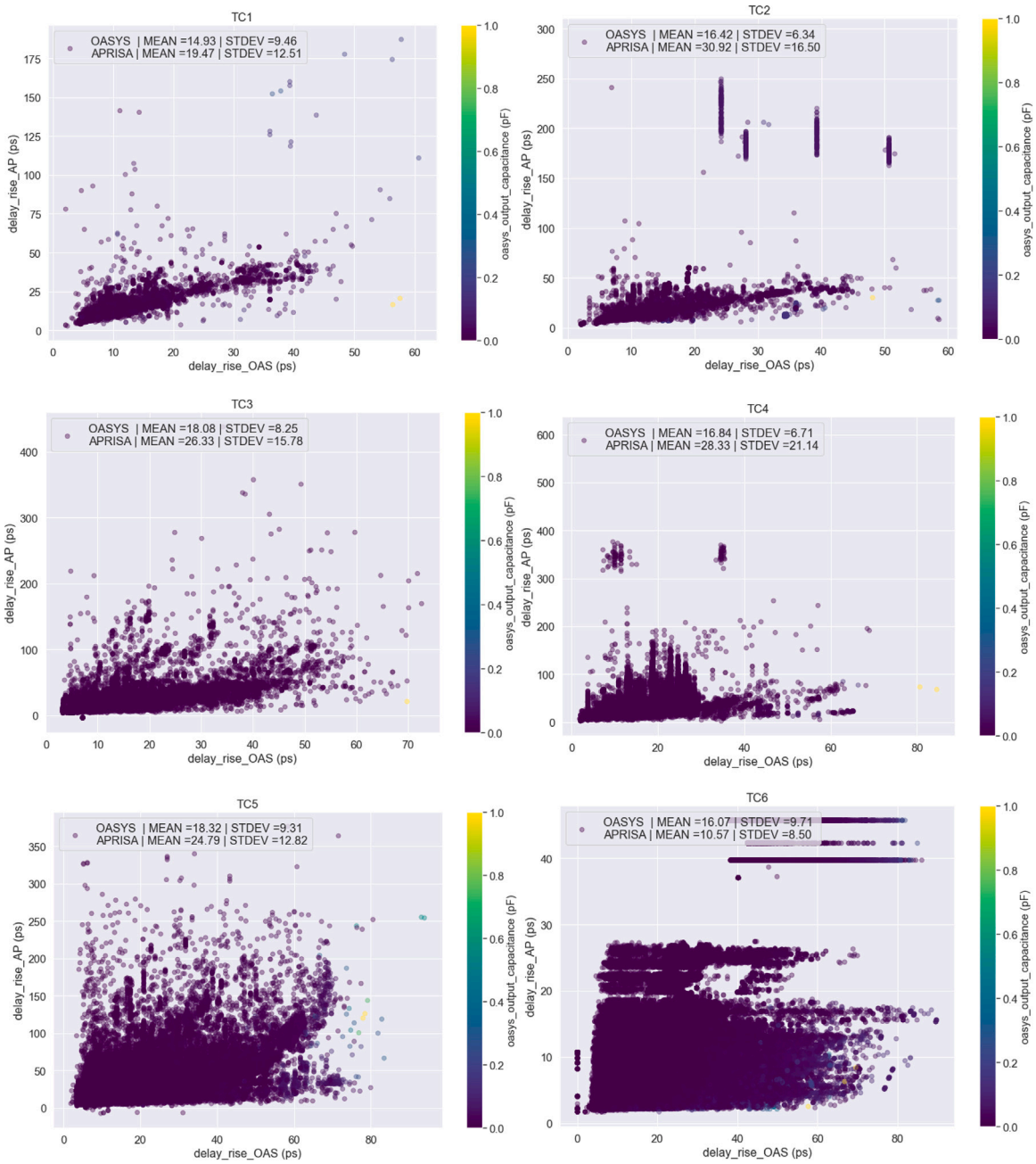

Fig. 4. Scatter plots depicting initial Oasys cell delay versus Aprisa cell delay.

Pandas, and Numpy libraries [19,20] to handle the master CSV file as a Dataframe. Table 3 summarizes the initial design information, and Table 5 for the refined dataframe after outliers removal.

The collected data is partitioned into distinct datasets for training-validation-testing. We have split TC1 to TC6 data as follows: 70% for training, 15% for validation, and 15% for testing. However, for TC7, we used the entire dataset for testing purposes.

In **Trial 1**, we performed individual model training for each design. Each design dataset was split into 70% for training, 15% for validation, and 15% for testing. This means that for each design, a separate

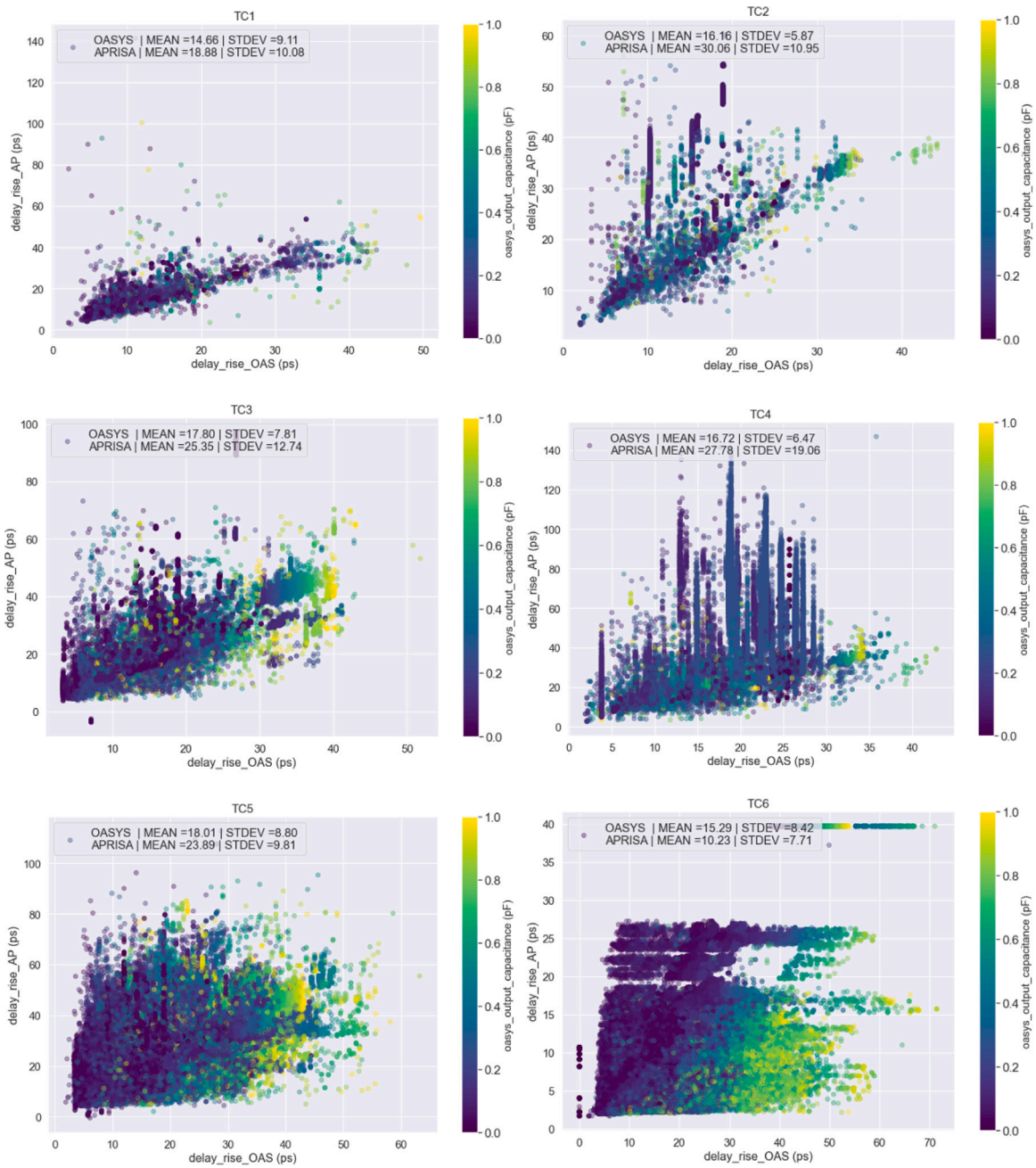

Fig. 5. Initial Oasys cell delay compared to golden Aprisa cell delay with outliers data removed.

model was trained using 70% of the data, validated using 15%, and tested on the remaining 15%. The purpose of this trial was to explore any potential dependencies between the design architecture and the performance of each machine learning algorithm (Table 2).

In **Trial 2**, we combined all six design datasets (TC1 to TC6) into one dataframe. This combined dataset was used to train six new models, one for each algorithm. The objective of this trial was to evaluate the generalization capability of each algorithm on unseen data. To accomplish this, we utilized TC7 as the sole testing dataset (Table 2).

Table 2
Training, validation and testing sets repartitioning per trial.

Trial	Training	Validation	Testing	Models count
I	70% TC1	15% TC1	15% TC1	5
	70% TC2	15% TC2	15% TC2	5
	70% TC3	15% TC3	15% TC3	5
	70% TC4	15% TC4	15% TC4	5
	70% TC5	15% TC5	15% TC5	5
	70% TC6	15% TC6	15% TC6	5
II	75% (TC1 to TC6)	25% (TC1 to TC6)	TC7	5

3.4. ML algorithms

The tool predicts delays for both positive and negative signal transitions, termed as *cell delay rise* and *cell delay fall* respectively. As a result, our initial focus was on training ML models to predict the *delay rise* attribute, which corresponds to positive unate arcs. Subsequently, we employed a similar approach to predict the *delay fall* attribute. Due to space limitations, this paper focuses on presenting the results achieved for the prediction of cell delay rise. However, the prediction of cell delay fall followed a similar approach.

Table 3
Design dataframes before outliers removal.

Design	Rows°				Columns°			Oasys delay			Aprisa delay	
	Training	Validation	Testing	Total	Int.	Float	Total	MEAN	STDEV	RMSE	MEAN	STDEV
TC1	3526	755	755	5036	34	11	45	14.93	9.46	17.654	19.47	12.51
TC2	52 731	17 577	17 578	87 886				16.42	6.34	17.568	30.92	16.50
TC3	56 947	18 983	18 983	94 913				18.08	8.25	19.850	26.33	15.78
TC4	115 207	38 403	38 403	192 013				16.84	6.71	18.100	28.33	21.14
TC5	570 744	190 248	190 248	951 240				18.32	9.31	20.524	24.79	12.82
TC6	1 094 066	364 689	364 689	1 823 444				16.07	9.71	18.764	10.57	8.50
Average	–	–	–	–	–	–	–	16.77	8.29	22.492	23.40	14.54

Machine Learning provides the capacity to unveil correlations among patterns and detect trends within intricate data structures. Our objective is to utilize ML techniques to unveil the relationships between the extracted cell delay and various other cell attributes. The predicted cell delay is a continuous numerical float value, which requires the use of supervised regression models. In our previous work [18], we applied various state-of-the-art regression models, including Linear Regression, Decision Tree, Random Forest, Support Vector Regression, and K-Nearest Neighbors. After evaluating their performance, we found that Random Forest had the highest predictive score.

Our current work introduces several advancements. Firstly, we expanded our exploration by applying ensemble methods Tree-based algorithms, extending the comparison beyond our previous best-performing model, the Random Forest. Secondly, we enhanced our analysis by incorporating two additional larger designs. Furthermore, in this study, we significantly refined the extracted features to a subset of 45 cell and net features. Finally, we tested the model's generalization capability using larger design (TC7). Table 1 delineates the size of each design in terms of the number of standard cells, highlighting the increased complexity and scale of the designs considered in this study.

The selected tree-based models include 'Extremely Randomized Forests (Extra-Trees)', 'Gradient Boosting (GB)', 'Light Gradient Boosting (LightGBM)', 'eXtreme Gradient Boosting (XGBoost)', and 'Adaptive Boosting (AdaBoost)' [21–23]. These algorithms are decision tree based structures, it partitions the inputs into regions. The internal nodes corresponds to features and each branch is called a decision. Each decision assigns a predicted value to each leaf fof the tree [24–26]. The Gradient Boosting techniques are built upon weak decision tree learners, it minimizes the errors of previous decision gradually until refining the overall model through the combination of weak learners. However, the Extra Trees (Extremely Randomized Trees) are ensemble methods that randomly select feature subsets for node and introduces additional randomness to mitigate variance and potential overfitting within the model [27,28].

To evaluate the quality of predictions, we utilize Root-Mean-Squared-Error and R2-score metrics. The RMSE function serves to assess the performance of each model by calculating the average magnitude of the errors produced, thereby providing a measure of the overall error rate. It is calculated by taking the square root of the mean of the squared differences between the predicted values and the actual values. A lower RMSE indicates better accuracy of the model. (Eq. (1)). Additionally, we employ the R2-score to evaluate the overall accuracy of the predictions. The R2 score, also known as the coefficient of determination, measures the variance in the target variable. A higher R2 score represents a more accurate alignment of the model predictions with the target values (Eq. (2)). In the next section, we compare each model's R2 and RMSE metrics and drew a conclusion [29].

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(pred_i - act_i)^2} \tag{1}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(act_i - pred_i)^2}{\sum_{i=1}^{n}(act_i - mean)^2} \tag{2}$$

3.5. Model features

Compared to our previous study, we have reduced the number of relevant cell-net features from 101 to 45, as most of the cell and net extracted attributes were not contributing during model prediction. The current study is conducted on a reduced number of features, this will help by gaining much training and inference runtime.

The collected datasets comprise a variety of categorical input variables, we have performed *feature encoding* by converting categorical columns into numerical integers and boolean types to integers.

As an example, the *Arc Sense* and *Arc Type* columns have a limited number of distinct values. We opted to treat these columns as *enumerate* type. Which involved creating new integer-type columns for each distinct value. The number of such additional columns corresponds to the count of distinct states. As a result, the dataframes result in a composition of 34 integer columns, 11 floating-point columns, and successful conversion of all categorical types into integers.

3.6. Dataset refinement

As a crucial step in data pre-processing, the removal of outliers is imperative to ensure the robustness and reliability of the dataset. Outliers refer to data points that deviate significantly from the majority of the dataset, often caused by miscalculations or other anomalies. In our approach, we began by plotting scatter plots for several floating-type attributes, including *cell output capacitance*, *input slew fall*, *output slew fall*, *input slew rise*, *output slew rise*, *delay rise*, *and delay fall*. These scatter plots allow us to visually identify any data points that appear as outliers.

Subsequently, we employed three different methods for outlier removal: "Z-score", "DBSCAN" and "Interquartile Range (IQR)". The focus is to get the highest accuracy while minimizing the number of removed outliers from the dataset. We have fine-tuned each method to identify the threshold beyond which the data is considered an outlier. We implemented Z-score at 3-4-5 standard deviation from the mean. Similarly, for IQR, we explored different quartile ranges, such as 75-25, 85-15, and 95-5, to identify the range at which outliers became prominent. We utilized the Interquartile Range (IQR) approach with a 4-sigma cutoff. This allowed us to establish up and low boundaries based on the mean value. Cells that fell beyond these bounds were considered outliers and subsequently discarded. Through iterative analysis, we discovered that the optimal approach involves initially applying the IQR method with a range of 85-15 to identify and discard the initial set of outliers that exceed the 4-sigma threshold. Subsequently, we proceeded to apply the Z-score method at the 4th standard deviation. Table 4 provides a summary of the percentage of outliers for each design.

The timing arcs that were considered outliers were removed from the TC1 to TC6 dataframes. While TC7 is considered intact raw data. In Fig. 5, we present a scatter plot after the removal of outliers, where the X-axis represents Oasys delays and the Y-axis represents the Aprisa delays. We included a heatmap visualization to highlight the output capacitance. The scattered dots in the plot illustrate the discrepancy between Oasys-RTL's cell delay compared to Aprisa delays.

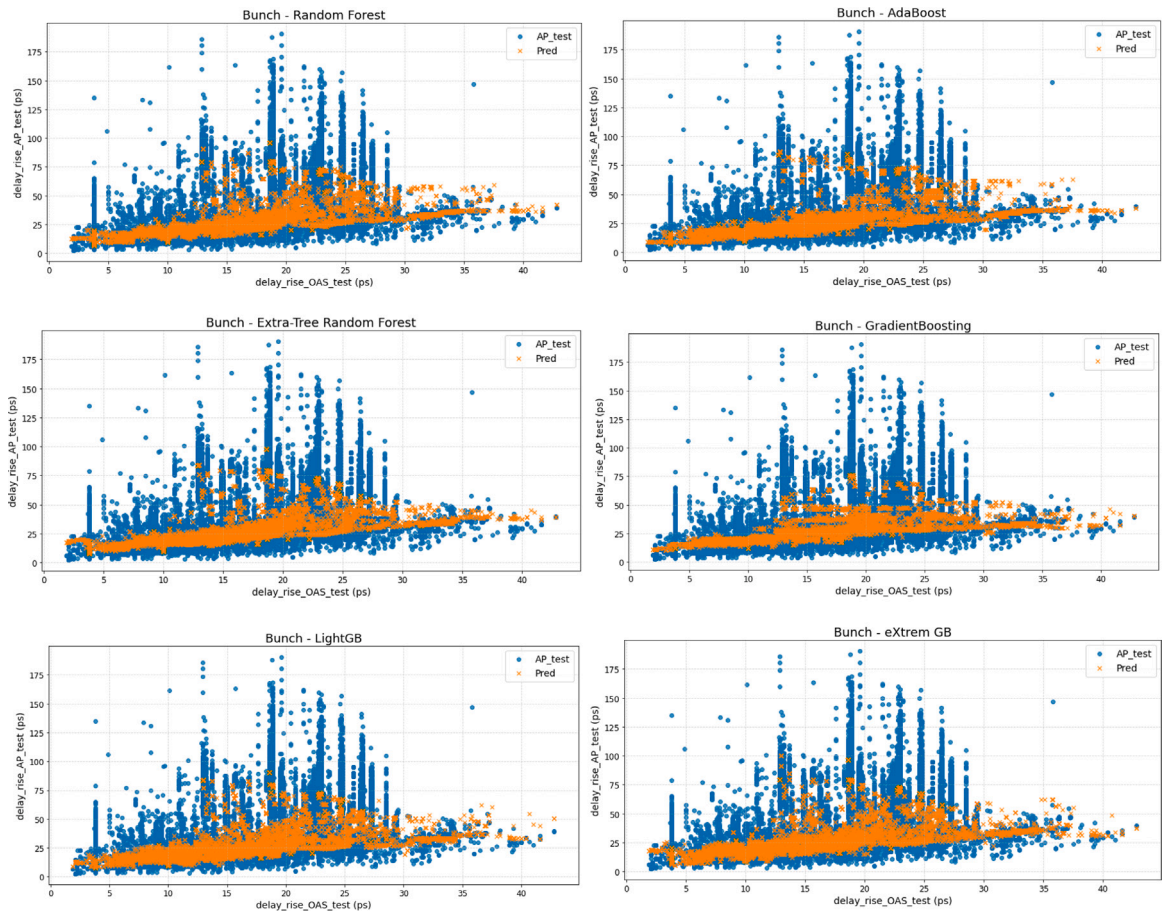

Fig. 6. Scatter plots comparing the predicted versus actual cell delays, employing TC7 testing data.

Table 4
Percentage of outliers per dataframe.

Test	Total	Outliers	Outliers (%)	Remaining
TC1	5036	102	2%	4934
TC2	87 886	2661	3%	85 225
TC3	94 913	2201	2.3%	92 712
TC4	192 013	2741	1.4%	189 272
TC5	951 240	24 132	2.5%	927 108
TC6	1 823 444	60 222	3.3%	1 763 222

Fig. 4 illustrates the initial delay comparison, including outliers, where certain outlier values significantly deviated from the expected data range, consequently impacting delay prediction. Subsequently, Fig. 6 displays refined delays after outlier removal, with noticeable alterations in the axis scales to accommodate these changes. Additionally, we remark a significant decrease in Root Mean Square Error (RMSE) following the removal of outliers as detailed in Tables 3 and 6. This notable reduction in RMSE emphasizes the substantial impact of outlier removal on enhancing the predictive accuracy of the model.

4. Experimental results

In the previous section, we presented a scattered plot comparing the initial Oasys delays with Aprisa delays (Fig. 5). The plot was generated after cleaning each design dataset by removing outliers. In this section, we focus on the results obtained after training, validating, and testing each model. We analyzed these results to identify the model that achieved the highest accuracy. Furthermore, we trained new models using the combined dataset from all test cases and evaluated their performance on unseen data.

4.1. Data analysis

The cell delays mean and standard deviation (stdev) are calculated and presented in Table 5. After refining the data and removing outliers, the initial mean and stdev are **16.44** and **7.74**, respectively. These values deviate considerably from Aprisa mean and stdev, which stand at **22.09** and **11.27**, respectively. To quantify the error in Oasys cell delays, we computed the root-mean-squared-error (RMSE), with the average initial Oasys RMSE being **12.119**, as shown in Table 5. This error metric provides insight into the overall accuracy of the Oasys estimations.

Next, we applied a 70-15-15 training-validation-testing split to each design from TC1 to TC6 using *train_test_split* function from Scikit-learn library. Each model was trained using the specific design dataset. The performance metrics are summarized in Table 6. The validation set was utilized to determine the optimal hyperparameters that yield the highest model accuracy. This process was carried out using the *GridSearchCV* function from the scikitlearn library. The purpose of this analysis was to examine how each model performs with different design architectures, referred as Trial 1 in Table 2. By training and evaluating the models on individual designs, we aimed to gain insights into their behavior and effectiveness in capturing the nuances of specific design characteristics.

Afterwards, the dataset was subjected to random shuffling and divided into a 75-25 training-validation split. The resulting combined dataset is denoted as “Bunch” and is referred to as such in Tables 7. To assess the generalization capability of the models, we evaluated their performance on new unseen data, specifically TC7. The models were exclusively tested on TC7 to gauge their ability to generalize beyond the training dataset. The R2 scores obtained on the testing set, using

Table 5
Design dataframes after outliers removal.

Design	Rows°				Columns°			Oasys delay			Aprisa delay	
	Training	Validation	Testing	Total	Int64	Float64	Total	MEAN	STDEV	RMSE	MEAN	STDEV
TC1	3453	741	740	4934	34	11	45	14.66	9.11	7.319	18.88	10.08
TC2	60 434	12 950	12 951	86 335				16.16	5.87	17.651	30.06	10.95
TC3	56 947	18 983	18 983	94 913				17.80	7.81	11.848	25.35	12.74
TC4	132 490	28 391	28 391	189 272				16.72	6.47	19.575	27.78	19.06
TC5	648 975	139 066	139 067	927 108				18.01	8.80	9.686	23.89	9.81
TC6	1 234 255	264 484	264 483	1 763 222				15.29	8.42	6.639	10.23	7.71
Average	–	–	–	–	–	–	–	**16.44**	**7.74**	**12.119**	**22.69**	**11.27**

Table 6
Models R2-scores and RMSE per design on testing dataset.

Design	RandomForest		AdaBoost		Extra-Trees		GradientBoosting		LightGB		XGBoost	
	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE
TC1	87.79	3.10	86.84	3.22	89.39	3.09	85.57	3.37	87.60	3.12	89.19	2.91
TC2	97.87	1.60	97.90	1.59	97.92	1.58	97.65	1.68	97.84	1.61	97.84	1.60
TC3	92.94	3.40	92.86	3.42	93.21	3.33	91.18	3.79	92.33	3.54	93.38	3.27
TC4	89.19	6.29	89.20	6.29	89.30	6.26	88.78	6.41	89.15	6.30	89.20	6.29
TC5	84.66	3.84	84.32	3.88	85.72	3.70	78.37	4.56	79.91	4.39	82.37	4.11
TC6	96.37	1.47	96.28	1.49	96.55	1.43	92.49	2.11	95.57	1.63	95.87	1.55
Average	**91.47**	**3.28**	91.23	3.31	**92.01**	**3.23**	89.00	3.65	90.40	3.43	90.66	3.28

Table 7
Bunch dataframe after outliers removal.

Design	Rows°				Columns°			Oasys delay			Aprisa delay	
	Training	Validation	Testing	Total	Int64	Float64	Total	MEAN	STDEV	RMSE	MEAN	STDEV
Bunch	2 313 105	771 035	–	3 084 140	34	11	45	16.427	8.635	9.686	16.687	12.483
TC7	–	–	1 441 448	1 441 448				16.705	8.846	18.890	10.833	6.570

Table 8
R2-scores and RMSE for models trained using Bunch dataset.

Design	RandomForest		AdaBoost		Extra-Trees		GradientBoosting		LightGB		XGBoost	
	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE	R2 (%)	RMSE
Bunch (TC7)	83.785	7.886	84.059	7.819	**84.266**	**7.768**	83.205	8.026	84.265	7.768	**84.321**	**7.754**

Fig. 7. Model's R2 comparison.

the best estimators identified by GridSearchCV on the validation set, are presented in Table 8.

4.2. Prediction results

Table 6 provides a summary of the accuracy scores achieved by each model and the corresponding RMSE. These scores were obtained after a hyper-parameter tuning process using GridSearch, which required significant computational runtime to identify the best estimator model parameters. Due to space limitations, we intentionally omitted discussing the intricate details of hyperparameter tuning and the validation scores. Instead, the focus was directed towards presenting the achieved scores utilizing the testing data and using best parameters per each model. It is noteworthy that the models' performance varies across different designs, reflecting the influence of the internal design architecture on the predictions. Notably, all models show improved accuracy compared to our previous work [18].

We observe that the top three models with the highest accuracy are *Extra-Trees Random Forest*, *Random Forest*, and *AdaBoost*, with average accuracies of 92.01%, 91.47%, and 91.23%, respectively. On the other hand, *XGBoost*, *LightGB*, and *GradientBoosting* show lower accuracy with average values of 90.66%, 90.40%, and 89.00%, respectively. These ensemble learning techniques, specifically based on gradient boosting, are less suitable for our data.

In Figs. 7 and 8, we present a visual comparison of the R2-score and RMSE for each model in the form of histograms. It is worth noting that

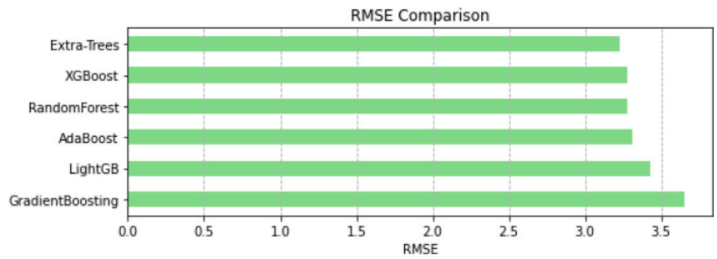

Fig. 8. Model's RMSE comparison.

Table 9
Training and inference time per model (in seconds).

Test	RandomForest		AdaBoost		Extra-Trees		GradientBoosting		LightGB		XGBoost	
	Train.	Inf.	Train.	Inf.	Train.	Inf.	Train.	Inf.	Train.	Inf.	Train.	Inf.
TC1	68	0.	2120	0.	30	0.	96	0	11	0.	56	0.
TC2	494	1	90	0.	235	1	2048	0.	10	0.	52	0.
TC3	1405	2	173	0.	462	2	1420	0.	9	0.	60	0.
TC4	1970	3	302	0.	794	3	2181	0.	10	0.	117	0.
TC5	21 049	19	2891	5	11 099	31	8321	4	1475	0.	573	0.
TC6	44 346	34	4436	6	21 572	86	21 665	7	56	0.	1376	0.
Bunch	54 350	–	11 426	–	37 962	–	29 401	–	41	–	1776	–
TC5	–	5.7	–	1.8	–	5.2	–	1.1	–	0.1	–	0.1

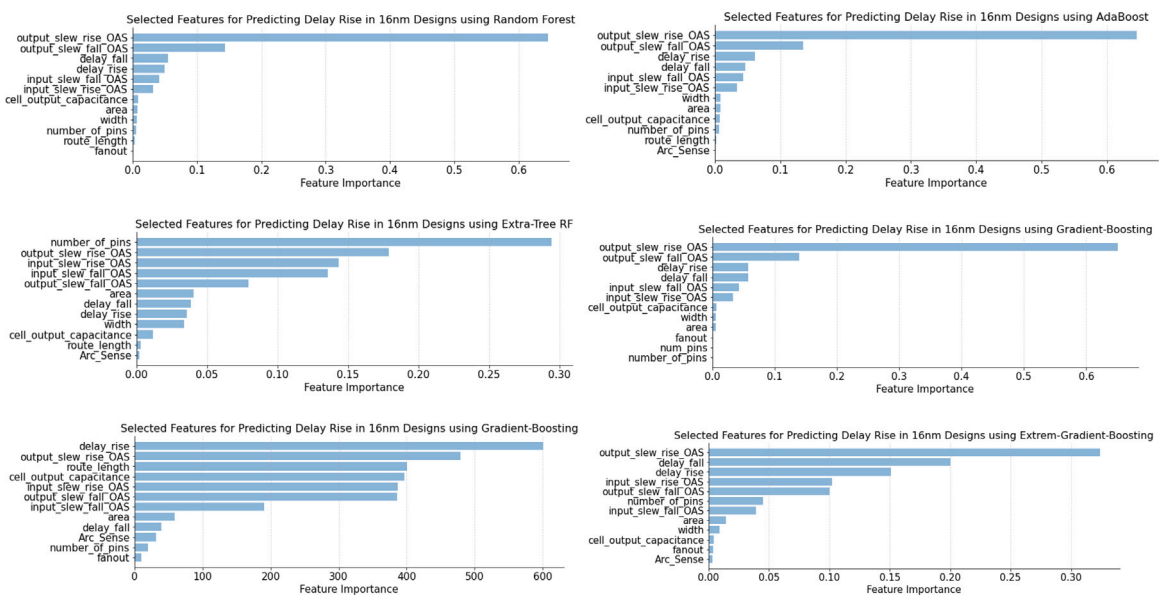

Fig. 9. Features selection for models trained using bunch dataset.

the error is inversely proportional to the R2-score. Among the models, *Extra-Trees Random Forest*, *Random Forest*, and *AdaBoost* exhibit the lowest RMSE values of 3.23, 3.28 and 3.31. On the other hand, *XGBoost*, *LightGB*, and *GradientBoosting* show higher errors with average values of 2.28, 3.43 and 3.65.

Similarly, the models were trained using the combined dataframe ('*Bunch*'), and tested using TC7 data. Among the models, the eXtreme Gradient Boosting model achieved the highest accuracy of **84.321**%, followed closely by the Extra-Tree RF with an accuracy of **84.266**%. It is important to note that the obtained scores for TC7 are lower than the average in Table 6. This is expected, as TC7 represents completely unseen data and might encompass outliers capable of influencing the model's predictive outcomes. However, despite the new data, the models still delivered a relatively high level of stability and predictive capability, with accuracy ranging between 83% and 84% for unseen data, which is quite promising. Fig. 6 portrays across six subplots the outcomes of TC7 testing set per model. The plots reveal a more cohesive distribution of predicted values, exhibiting enhanced performance as they align closely with the golden value.

Even though the *eXtreme Gradient Boosting* and *Extra-Tree Random Forest* models achieved high scores when trained using the Bunch dataset, we also considered their training and inference times. The Extra-Tree RF model took around 10 h to train and predict cell delays, as shown in Table 9, while the *eXtreme GB* model only required 30 min for the same task. On the other hand, the *Random Forest* model took longer, lasting around 15 h, therefore, considering the balance between model prediction accuracy and runtime, the *eXtreme Gradient Boosting* and *Extra-Tree Random Forest* models seem more practical choices for our application. All models were executed on the same machine with identical CPU and memory configurations.

Ultimately, we conducted feature selection on models trained with Bunch dataset to emphasize the significant attributes influencing delay prediction. In Fig. 9, we present the selected features per model basis. During training process, we initially considered a pool of 45 features. However, for the sake of clarity, the complete set of features is not displayed in Fig. 9. Instead, we exclusively highlights the subset of features that exhibited higher contributions in the delay prediction.

5. Conclusion

In this paper, we focused on applying Ensemble methods and Tree-based techniques to enhance the accuracy of cell delays in a post-placed netlist. Our approach involved training and testing six ML models using 16 nm industrial designs, leveraging the Aprisa backend information as golden data for comparison. Through our proposed models, we achieved higher accuracy and lower error rates, with the "Extra-Tree Random Forest" and "Random Forest" models exhibiting the highest accuracy among all models. However, in terms of generalization capability on unseen data, the "Extra-Tree Random Forest" as well as the "Extreme Gradient Boosting" models demonstrated the highest prediction accuracy.

Using Ensemble methods, combining Tree-based and Gradient-based algorithms, we successfully reduced the average RMSE from **12.119** to **3.23**, accompanied by an impressive average accuracy of **92.01**%. Both the "Extra-Tree RF" and "Extreme GB" models show higher stability and predictive capability on new unseen testing data, achieving an average accuracy of **84.26**% lowering the average RMSE to **7.768**.

With this model, precise delay estimations can be obtained without relying on physical information, utilizing only attributes within the Oasys synthesis tool and delivering accurate delays within a short inference time. As extension of this work, we aim to improve the training set by incorporating supplementary data corresponding to different operating frequencies of same designs, enhancing the model's overall performance and versatility.

During the preparation of this work the author(s) used OpenAI ChatGPT service in order to improve language and readability. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

CRediT authorship contribution statement

Yassine Attaoui: Conception and design of study, Acquisition of data, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing. **Mohamed Chentouf:** Conception and design of study, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing. **Zine El Abidine Alaoui Ismaili:** Conception and design of study, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing. **Aimad El Mourabit:** Conception and design of study, Analysis and/or interpretation of data, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

All authors approved the version of the manuscript to be published.

References

[1] C. Mohamed, M. Soukaina, A.I.Z. El Abidine, Power aware setup timing optimization in physical design of ASICs, Microelectron. J. 83 (2019) 147–154.
[2] Y. Attaoui, M. Chentouf, Z.E.A.A. Ismaili, A. El Mourabit, A new MBFF merging strategy for post-placement power optimization of IoT devices, in: 2021 IEEE/ACS 18th International Conference on Computer Systems and Applications, AICCSA, IEEE, 2021, pp. 1–6.
[3] Y. Attaoui, M. Chentouf, Z.E.A.A. Ismaili, A. El Mourabit, Clock gating efficiency and impact on power optimization during synthesis flow, in: 2021 International Conference on Microelectronics, ICM, IEEE, 2021, pp. 13–16.
[4] Siemens EDA oasys-RTL™ user's guide, 2021.
[5] Siemens EDA APRISA™ user's guide, 2022.
[6] S.-S. Han, A.B. Kahng, S. Nath, A.S. Vydyanathan, A deep learning methodology to proliferate golden signoff timing, in: 2014 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2014, pp. 1–6.
[7] E.C. Barboza, N. Shukla, Y. Chen, J. Hu, Machine learning-based pre-routing timing prediction with reduced pessimism, in: 2019 56th ACM/IEEE Design Automation Conference, DAC, IEEE, 2019, pp. 1–6.
[8] A.B. Kahng, S. Kang, H. Lee, S. Nath, J. Wadhwani, Learning-based approximation of interconnect delay and slew in signoff timing tools, in: 2013 ACM/IEEE International Workshop on System Level Interconnect Prediction, SLIP, IEEE, 2013, pp. 1–8.
[9] A.B. Kahng, U. Mallappa, L. Saul, Using machine learning to predict path-based slack from graph-based timing analysis, in: 2018 IEEE 36th International Conference on Computer Design, ICCD, IEEE, 2018, pp. 603–612.
[10] W.-T.J. Chan, K.Y. Chung, A.B. Kahng, N.D. MacDonald, S. Nath, Learning-based prediction of embedded memory timing failures during initial floorplan design, in: 2016 21st Asia and South Pacific Design Automation Conference, ASP-DAC, IEEE, 2016, pp. 178–185.
[11] R. Liang, Z. Xie, J. Jung, V. Chauha, Y. Chen, J. Hu, H. Xiang, G.-J. Nam, Routing-free crosstalk prediction, in: 2020 IEEE/ACM International Conference on Computer Aided Design, ICCAD, IEEE, 2020, pp. 1–9.
[12] M. Chentouf, C. Naimy, Z.E.A.A. Ismaili, Machine learning application for early power analysis accuracy improvement: A case study for cells switching power, in: 2021 International Conference on Microelectronics, ICM, IEEE, 2021, pp. 17–20.
[13] Z. Guo, M. Liu, J. Gu, S. Zhang, D.Z. Pan, Y. Lin, A timing engine inspired graph neural network model for pre-routing slack prediction, 2022.
[14] A.B. Kahng, M. Luo, S. Nath, SI for free: machine learning of interconnect coupling delay and transition effects, in: 2015 ACM/IEEE International Workshop on System Level Interconnect Prediction, SLIP, IEEE, 2015, pp. 1–8.
[15] Z. Wang, S. Liu, Y. Pu, S. Chen, T.-Y. Ho, B. Yu, Restructure-tolerant timing prediction via multimodal fusion, in: 2023 60th ACM/IEEE Design Automation Conference, DAC, IEEE, 2023, pp. 1–6.
[16] X. He, Z. Fu, Y. Wang, C. Liu, Y. Guo, Accurate timing prediction at placement stage with look-ahead RC network, in: Proceedings of the 59th ACM/IEEE Design Automation Conference, 2022, pp. 1213–1218.
[17] O.S. Ram, S. Saurabh, Modeling multiple-input switching in timing analysis using machine learning, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 40 (4) (2020) 723–734.
[18] Y. Attaoui, M. Chentouf, Z.E.A.A. Ismaili, A. El Mourabit, Machine learning application for cell delay accuracy improvement at post-placement stage: A case study for combinational cells, Integration (2023).
[19] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and Tensorflow: Concepts, Tools, and Techniques to Build Intelligent Systems, " O'Reilly Media, Inc.", 2019.
[20] G. Hackeling, Mastering Machine Learning with Scikit-Learn, Packt Publishing Ltd, 2017.
[21] scikit-learn contributors, scikit-learn: Ensemble methods, https://scikit-learn.org/stable/modules/ensemble.html#gradient-tree-boosting.
[22] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.
[23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: A highly efficient gradient boosting decision tree, Adv. Neural Inf. Process. Syst. 30 (2017).
[24] API reference — scikit-learn.org, 2023, https://scikit-learn.org/stable/modules/classes.html#module-sklearn.ensemble, [Accessed 19-11-2023].
[25] Sklearn.ensemble.RandomForestRegressor — scikit-learn.org, 2023, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html#sklearn.ensemble.RandomForestRegressor, [Accessed 19-11-2023].
[26] Sklearn.ensemble.AdaBoostRegressor — scikit-learn.org, 2023, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostRegressor.html#sklearn.ensemble.AdaBoostRegressor, [Accessed 19-11-2023].
[27] Sklearn.ensemble.ExtraTreesRegressor — scikit-learn.org, 2023, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesRegressor.html#sklearn.ensemble.ExtraTreesRegressor, [Accessed 19-11-2023].
[28] Sklearn.ensemble.GradientBoostingRegressor — scikit-learn.org, 2023, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html#sklearn.ensemble.GradientBoostingRegressor, [Accessed 19-11-2023].
[29] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in python, J. Mach. Learn. Res. 12 (2011) 2825–2830.